

DMA Controller

Design Reference — Signal Lines, Buses & Transfer Flow

AHB/APB Bus System · 32-bit RISC-V · Custom DMA

1. Overview — What a DMA Does and Why

A Direct Memory Access (DMA) controller moves blocks of data between memory and peripherals without keeping the CPU busy for every byte. The CPU programs the DMA once, then goes back to executing code while the DMA handles the data movement autonomously.

Without DMA: the CPU must read each byte from a source address and write it to a destination address, one word at a time — burning thousands of cycles on data movement instead of computation.

With DMA: the CPU writes five register values to the DMA controller, sets the enable bit, then walks away. The DMA handles everything — bus arbitration, read cycles, write cycles, address incrementing, beat counting — and interrupts the CPU only when the entire block is done.

Three separate conversations happen in every DMA transfer: ① APB (CPU → DMA config registers) ② AHB (DMA ↔ Memory/Peripherals for data) ③ Sideband (DMA ↔ Peripheral handshake + IRQ to CPU)

2. The Three Bus Interfaces

2.1 APB — Advanced Peripheral Bus (Configuration)

APB is a simple, low-power bus used only for register access. It is NOT used to move bulk data. The CPU is the APB master. The DMA is an APB slave. The CPU uses APB to write the DMA's internal configuration registers before a transfer begins.

Key concept: APB is a 2-cycle protocol. Every access takes exactly two clock cycles: Setup cycle (PENABLE=0) then Enable cycle (PENABLE=1). The DMA only captures data on the rising edge of the Enable cycle. This gives the slave a full cycle to prepare, eliminating glitches.

APB Signal Table

Signal	Direction	Value / State	Role & Why It Exists
PSEL	CPU → DMA	0 / 1	Chip-select for this specific slave. Each APB slave has its own PSEL. Without it the DMA ignores the bus completely.
PENABLE	CPU → DMA	0 → 1	The 2-cycle enable. Cycle 1: PENABLE=0 (setup). Cycle 2: PENABLE=1. DMA latches PWDATA only when PENABLE=1 and PREADY=1 together.
PWRITE	CPU → DMA	1=write, 0=read	Tells DMA which direction: 1=CPU writing to DMA register, 0=CPU reading back from DMA register.
PADDR	CPU → DMA	0x00–0x14	Register select. Decoded by DMA to pick which internal register to read/write. 0x00=SRC_ADDR, 0x04=DST_ADDR, 0x08=LEN, 0x0C=CTRL, 0x10=STATUS, 0x14=INT_CLR.
PWDATA	CPU → DMA	32-bit value	The data the CPU is writing to the selected register. e.g. 0x20001000 written to SRC_ADDR sets the DMA source pointer.
PRDATA	DMA → CPU	register value	Data the DMA returns when CPU reads (PWRITE=0). CPU reads STATUS to check BUSY/DONE/ERR flags, or reads back SRC_ADDR to verify.
PREADY	DMA → CPU	0 = wait, 1 = done	Slave stall signal. DMA holds PREADY=0 if it needs extra cycles to process an access. CPU's APB controller inserts wait states and freezes until PREADY=1. Simple implementations tie this to 1 permanently.

DMA Internal Registers (programmed by CPU over APB)

These are flip-flop registers inside the DMA chip — not memory, not CPU registers. They are the DMA's own notepad. PADDR selects which one the CPU is accessing.

PADDR offset	Register	Contents & Purpose
0x00	SRC_ADDR	32-bit source address. DMA reads data FROM this address. e.g. 0x20001000 (SRAM buffer).
0x04	DST_ADDR	32-bit destination address. DMA writes data TO this address. e.g. 0x40004400 (UART TX FIFO).
0x08	LEN	Number of beats (transfers) to perform. DMA decrements an internal counter from this value to 0.
0x0C	CTRL	Control bits: [0]=EN (start), [2:1]=direction, [4:3]=size(byte/half/word), [5]=burst enable, [6]=IRQ enable.
0x10	STATUS	Read-only flags: [0]=BUSY (transfer in progress), [1]=DONE (completed), [2]=ERR (HRESP error occurred).
0x14	INT_CLR	CPU writes 1 here in its ISR to clear the IRQ and reset DONE/ERR flags, returning DMA to IDLE.

2.2 AHB — Advanced High-performance Bus (Data Movement)

AHB is the fast bus. This is where bulk data actually moves. The DMA acts as an AHB master — it drives addresses and controls transfers. Memory and peripherals are AHB slaves — they respond to the DMA's requests.

AHB is a pipelined bus: the address for the NEXT beat is placed on the bus one cycle before the data of the CURRENT beat is available. This keeps the bus 100% utilized during a burst.

Bus arbitration: The AHB bus is shared between the CPU and DMA. Before driving any signals, the DMA must request the bus (HBUSREQ=1) and wait for the arbiter to grant it (HGRANT=1). Only then can the DMA drive HADDR and HTRANS.

AHB Arbitration Signals

Signal	Direction	Value / State	Role & Why It Exists
HBUSREQ	DMA → Arbiter	0 → 1	DMA requests ownership of the AHB bus. Held high throughout the entire transfer. DMA must NOT drive HADDR until HGRANT=1.
HGRANT	Arbiter → DMA	0 → 1	Arbiter gives DMA permission to use the bus. If CPU holds the bus, DMA

Signal	Direction	Value / State	Role & Why It Exists
			waits here. DMA FSM stays in IDLE until this is asserted.
HLOCK	DMA → Arbiter	1 (optional)	Locks the bus for atomic multi-beat bursts. Prevents the arbiter from reassigning the bus mid-burst, ensuring data coherency.

AHB Address-Phase Signals (DMA drives these)

Every AHB transfer has an address phase where the master (DMA) describes what it wants to do, followed by a data phase where the slave responds.

Signal	Direction	Value / State	Role & Why It Exists
HADDR[31:0]	DMA → Bus	<code>src_ptr / dst_ptr</code>	32-bit address for this beat. During reads: source address (where to fetch data). During writes: destination address (where to put data). Incremented by HSIZE bytes after each successful beat.
HTRANS[1:0]	DMA → Bus	2'b10 NONSEQ	Transfer type. IDLE (00)=no transfer. BUSY (01)=master pausing a burst. NONSEQ (10)=first beat or single transfer. SEQ (11)=continuation of an ongoing burst. Slaves check this every cycle.
HWRITE	DMA → Bus	0=read, 1=write	Direction of THIS beat. 0: DMA is reading from HADDR (source fetch). 1: DMA is writing to HADDR (destination store). Changes each time DMA switches between read and write phases.
HSIZE[2:0]	DMA → Bus	3'b010 = word	Transfer width. 000=byte (8-bit), 001=halfword (16-bit), 010=word (32-bit). Slaves use this to know how many bytes to supply or capture. Set from CTRL register configuration.
HBURST[2:0]	DMA → Bus	3'b001 = INCR	Burst descriptor. SINGLE (000)=one beat only. INCR (001)=incrementing burst, length unspecified. INCR4/WRAP4 etc. tell slaves exactly how many beats follow, allowing them to pre-fetch or optimize internally.

Signal	Direction	Value / State	Role & Why It Exists
HPROT[3:0]	DMA → Bus	4'b0011	Protection/privilege hint. Bit0: privileged(1) vs user(0). Bit1: data(1) vs instruction(0). Bits3:2: cacheable/bufferable. DMA drives privileged+data (0011). Slaves may use this for access-control checks.

AHB Data-Phase Signals (Slave responds)

Signal	Direction	Value / State	Role & Why It Exists
HRDATA[31:0]	Slave → DMA	actual data	The 32-bit word the slave returns for a read. Valid only when HREADY=1. DMA latches this into its internal data_buf register on that clock edge.
HWDATA[31:0]	DMA → Bus	data_buf	Data DMA sends to the destination slave for a write. Driven from the internal data_buf that was filled during the preceding read phase.
HREADY	Slave → DMA	0=wait, 1=done	The most critical timing signal. 1: slave has finished, data is valid (for reads) or captured (for writes). 0: slave needs more time — DMA holds its address and enters WAIT_HREADY state, inserting wait cycles.
HRESP[1:0]	Slave → DMA	00=OKAY	Transfer status. OKAY (00)=success. ERROR (01)=slave reports a fault (bad address, protection violation). On ERROR, DMA sets STATUS.ERR=1 and fires IRQ so CPU can handle it.

2.3 Sideband — Peripheral Handshake & Interrupt

These signals have nothing to do with AHB or APB. They are direct wires between the DMA and peripherals/CPU, used to coordinate who is ready and when a transfer finishes.

Signal	Direction	Value / State	Role & Why It Exists
dma_req[n]	Peripheral → DMA	0 → 1	Peripheral asserts this when it is ready for a transfer. e.g. UART TX FIFO half-empty triggers

Signal	Direction	Value / State	Role & Why It Exists
			<code>dma_req[0]=1</code> . DMA's channel arbiter sees this and schedules a transfer on that channel.
<code>dma_ack[n]</code>	DMA → Peripheral	pulse 1 clk	DMA acknowledges the request. Peripheral lowers <code>dma_req</code> after seeing <code>ack</code> , preventing re-triggering while the transfer is running.
<code>dma_done[n]</code>	DMA → Peripheral	pulse 1 clk	Fired when <code>len_cnt</code> reaches 0 — the entire block is done. Peripheral knows it can act on the received data. Also triggers the IRQ to the CPU.
IRQ	DMA → CPU	1	Interrupt to the CPU's interrupt controller. Forces CPU to jump to the DMA ISR. CPU then reads STATUS register, handles errors, programs the next transfer if needed, and clears the interrupt.

3. Transfer Flow — Phase by Phase

A complete memory-to-peripheral DMA transfer walks through these seven phases. Each phase is shown with the active signals and what is happening inside the DMA's FSM.

Phase 1 — CPU Programs the DMA (APB Write)

The CPU writes five register values to the DMA over APB. This is not data movement — it is configuration. The DMA's FSM remains in IDLE throughout this phase. `HBUSREQ` stays 0; `AHB` is untouched.

- `PSEL=1, PWRITE=1, PADDR=0x00, PWDATA=SRC_ADDR` → DMA latches source address on `PENABLE` edge
- `PSEL=1, PWRITE=1, PADDR=0x04, PWDATA=DST_ADDR` → DMA latches destination address
- `PSEL=1, PWRITE=1, PADDR=0x08, PWDATA=N` → DMA latches beat count into `len_cnt` register
- `PSEL=1, PWRITE=1, PADDR=0x0C, PWDATA=0x41` → writes CTRL: `EN=1, IRQ_EN=1, SIZE=word`
- On `EN` bit set: DMA FSM transitions from IDLE → `BUS_REQ`. APB phase complete.

Phase 2 — DMA Requests the AHB Bus

DMA raises HBUSREQ and waits for the arbiter to grant the bus. If the CPU is actively using AHB, the DMA waits here. The CPU's own bus cycles are not affected — it continues fetching instructions.

- DMA asserts HBUSREQ=1 to the bus arbiter
- DMA drives HTRANS=IDLE (does NOT yet drive any address)
- Arbiter waits for the CPU to finish its current AHB transaction
- Arbiter asserts HGRANT=1 to the DMA
- DMA FSM sees HGRANT=1 → transitions to ADDR_PHASE

Phase 3 — Peripheral Triggers the Channel (Sideband)

For peripheral-driven transfers, the peripheral signals readiness via dma_req before or during the bus request phase. The DMA's channel arbiter selects which channel to service.

- Peripheral asserts dma_req[n]=1 (UART FIFO is ready, ADC conversion complete, etc.)
- DMA channel arbiter latches ch_req[n]=1 in internal register
- Arbiter selects channel n (round-robin or fixed priority)
- DMA asserts dma_ack[n]=1 for one clock cycle
- Peripheral lowers dma_req after seeing ack. DMA loads src_ptr and dst_ptr from channel n registers.

Phase 4 — Address Phase (AHB)

DMA drives the source address and describes the transfer. Slaves see this address and prepare their response. Due to AHB pipelining, the data from this address will not appear until the NEXT clock cycle.

- DMA drives HADDR = src_ptr (current source address)
- DMA drives HTRANS = NONSEQ (first beat) or SEQ (subsequent burst beat)
- DMA drives HWRITE = 0 (this is a read from source)
- DMA drives HSIZE = 010 (word), HBURST = INCR, HPROT = 0011
- Memory slave sees its address on HADDR and starts preparing data internally

Phase 5 — Data Phase: Read (AHB)

One cycle after the address phase, the memory slave drives read data back on HRDATA. DMA waits for HREADY=1 before latching. If the slave is slow (SRAM wait states, busy flash), it holds HREADY=0 and the DMA inserts wait cycles.

- Slave drives HRDATA with the 32-bit word at the requested address
- If HREADY=0: DMA holds current address, enters WAIT_HREADY, inserts wait states
- When HREADY=1 and HRESP=OKAY: DMA latches HRDATA into internal data_buf register
- DMA increments src_ptr by 4 (word size) for next beat
- DMA immediately starts the write address phase in the same cycle (AHB pipeline)

Phase 6 — Data Phase: Write (AHB)

DMA switches to write master. It drives the destination address, sets HWRITE=1, and places the buffered data on HWDATA. The destination slave captures HWDATA when it asserts HREADY=1.

- DMA drives HADDR = dst_ptr (destination address, e.g. UART TX data register)
- DMA drives HWRITE = 1, HSIZE = 010, HTRANS = NONSEQ
- DMA drives HWDATA = data_buf (the word captured in Phase 5)
- Destination slave captures HWDATA on its HREADY rising edge
- DMA increments dst_ptr, decrements len_cnt. If len_cnt > 0: return to Phase 4. If len_cnt = 0: proceed to Phase 7.

Phase 7 — Transfer Complete

When len_cnt reaches 0 after a successful write, the DMA is done. It releases the bus, signals the peripheral, and interrupts the CPU.

- DMA drives HTRANS = IDLE (releases bus activity)
- DMA lowers HBUSREQ = 0 (releases bus ownership to arbiter)
- DMA pulses dma_done[n] = 1 for one cycle
- DMA asserts IRQ = 1 to the CPU interrupt controller
- DMA sets STATUS.DONE=1, STATUS.BUSY=0 in internal register
- CPU ISR fires: reads STATUS, programs next transfer or clears with INT_CLR write
- DMA FSM returns to IDLE on EN=0

4. Internal DMA Registers (Hardware, not APB-visible)

These are flip-flops inside the DMA's datapath, not accessible over APB. They are the DMA's working memory during a transfer.

Register	Purpose
src_ptr	Current source read pointer. Initialized from SRC_ADDR register. Auto-incremented by HSIZE bytes after each successful read beat.
dst_ptr	Current destination write pointer. Initialized from DST_ADDR register. Auto-incremented by HSIZE bytes after each successful write beat.
len_cnt	Remaining beat counter. Initialized from LEN register. Decrementing after each complete read+write pair. Transfer ends when this reaches 0.

Register	Purpose
<code>data_buf</code>	Temporary 32-bit buffer holding data between the read phase (latched from HRDATA) and the write phase (driven onto HWDATA). One word wide for single-beat transfers.
<code>ch_sel</code>	Active channel index (0 to N-1). Set by the channel arbiter when it picks which channel's <code>dma_req</code> to service.
<code>err_flag</code>	Set to 1 if HRESP=ERROR is received during any beat. Propagated to STATUS.ERR register for CPU to read.

5. FSM State Summary

The DMA controller's finite state machine (FSM) drives all signal transitions. Each state corresponds to a distinct phase of bus activity.

State	Signals Active	What Happens
IDLE	<code>HBUSREQ=0</code> , <code>HTRANS=IDLE</code>	Waiting for <code>dma_req</code> from a peripheral or EN bit set. No bus activity. APB register writes are accepted.
BUS_REQ	<code>HBUSREQ=1</code> , <code>HTRANS=IDLE</code>	DMA requests the bus. Waits for <code>HGRANT=1</code> from arbiter. CPU may still be using AHB.
ADDR_PHASE	<code>HADDR</code> , <code>HTRANS</code> , <code>HWRITE</code> , <code>HSIZE</code> , <code>HBURST</code> driven	DMA drives the address and transfer descriptor for the source read. Slave sees the address and prepares.
READ_DATA	Sampling <code>HRDATA</code>	Waiting for <code>HREADY=1</code> . When ready, latches <code>HRDATA</code> into <code>data_buf</code> . If <code>HREADY=0</code> , stays here (inserts wait states).
WRITE_DATA	<code>HADDR=dst_ptr</code> , <code>HWRITE=1</code> , <code>HWDATA=data_buf</code>	Writes buffered data to destination. Waits for <code>HREADY=1</code> from destination. Decrements <code>len_cnt</code> .
WAIT_HREADY	All signals held stable	Slave asserted <code>HREADY=0</code> . DMA freezes current address and data on the bus until slave is ready.
DONE	<code>IRQ=1</code> , <code>dma_done=1</code> , <code>HBUSREQ=0</code> , <code>HTRANS=IDLE</code>	<code>len_cnt=0</code> . Transfer complete. Fires interrupt. Releases bus. Waits for CPU to clear <code>CTRL.EN</code> .

6. Quick Reference — Complete Signal Listing

All signals in one place, grouped by interface.

APB Slave Interface (Configuration)

Signal	Direction	Value / State	Role & Why It Exists
PCLK	System → DMA	clock	APB clock. All APB signals sampled on rising edge.
PRESETn	System → DMA	active-low	APB reset. Low = reset all config registers to default.
PSEL	CPU → DMA	0/1	Select this DMA slave on the APB.
PENABLE	CPU → DMA	0→1 each cycle	Enable strobe. Second cycle of every APB transaction.
PWRITE	CPU → DMA	0/1	1=write to DMA register. 0=read from DMA register.
PADDR	CPU → DMA	32-bit	Register address offset into DMA register map.
PWDATA	CPU → DMA	32-bit	Write data from CPU to selected DMA register.
PRDATA	DMA → CPU	32-bit	Read data from DMA register to CPU.
PREADY	DMA → CPU	0/1	0=DMA needs more time. 1=access complete.
PSLVERR	DMA → CPU	0/1	Optional. 1=access error (bad address etc.).

AHB Master Interface (Data Movement)

Signal	Direction	Value / State	Role & Why It Exists
HCLK	System → DMA	clock	AHB clock. All AHB signals sampled on rising edge.
HRESETn	System → DMA	active-low	AHB reset.
HBUSREQ	DMA → Arbiter	0/1	DMA requests the AHB bus. Held high during transfer.
HGRANT	Arbiter → DMA	0/1	1=DMA owns the bus and may drive HADDR/HTRANS.
HLOCK	DMA → Arbiter	0/1	1=DMA holds bus locked for an atomic burst.

Signal	Direction	Value / State	Role & Why It Exists
HADDR	DMA → Bus	32-bit	Address driven by DMA. Source or destination depending on phase.
HTRANS	DMA → Bus	2-bit	00=IDLE, 10=NONSEQ, 11=SEQ. Tells slaves if transfer is active.
HWRITE	DMA → Bus	0=read, 1=write	0 during source read phase. 1 during destination write phase.
HSIZE	DMA → Bus	3-bit	000=byte, 001=halfword, 010=word. Transfer granularity.
HBURST	DMA → Bus	3-bit	000=SINGLE, 001=INCR, 010=WRAP4, 011=INCR4, etc.
HPROT	DMA → Bus	4-bit	Privilege/cache hints. Typically 0011 (privileged data).
HWDATA	DMA → Bus	32-bit	Write data from DMA to destination slave.
HRDATA	Slave → DMA	32-bit	Read data from source slave to DMA.
HREADY	Slave → DMA	0/1	0=insert wait state. 1=transfer complete, proceed.
HRESP	Slave → DMA	2-bit	00=OKAY, 01=ERROR. DMA sets err_flag on ERROR.

Sideband Handshake

Signal	Direction	Value / State	Role & Why It Exists
dma_req[N-1:0]	Peripheral → DMA	per-channel	N-bit bus, one bit per DMA channel. Peripheral asserts its bit when ready for a transfer.
dma_ack[N-1:0]	DMA → Peripheral	1-cycle pulse	DMA acknowledges the request. Peripheral de-asserts dma_req after seeing its ack bit.
dma_done[N-1:0]	DMA → Peripheral	1-cycle pulse	Transfer complete for that channel. Entire LEN-word block has been moved.
IRQ	DMA → CPU	level/pulse	Interrupt to CPU interrupt controller. CPU ISR reads STATUS register to determine cause.

End of Document